

Algoritmos Heurísticos para Árvore de Steiner: Visualização Interativa em 3D

Arthur Lunas¹, Paulo Felipe¹, Sidiney Barbosa¹

¹Instituto Federal Goiano — Campus Rio Verde
Rodovia Sul Goiana, km 01, Zona Rural, 75901-970, Rio Verde, Goiás, Brasil.

arthur.barbosa@estudante.ifgoiano.edu.br

paulo.cazon@estudante.ifgoiano.edu.br

sidiney.barbosa@estudante.ifgoiano.edu.br

Abstract. *This paper presents the development of a 3D interactive graph visualization tool for exploring Steiner tree heuristic algorithms. The implementation uses Three.js to create an immersive environment where users can construct graphs, select terminals, and visualize the execution of Takahashi-Matsuyama and Shortest Path Heuristic (SPH) algorithms. The tool provides real-time feedback, allowing comparison between different heuristic approaches and understanding of their behavior through visual representation.*

Resumo. *Este artigo apresenta o desenvolvimento de uma ferramenta interativa de visualização de grafos em 3D para exploração de algoritmos heurísticos de Árvore de Steiner. A implementação utiliza Three.js para criar um ambiente imersivo onde usuários podem construir grafos, selecionar terminais e visualizar a execução dos algoritmos Takahashi-Matsuyama e Shortest Path Heuristic (SPH). A ferramenta fornece feedback em tempo real, permitindo a comparação entre diferentes abordagens heurísticas e a compreensão de seu comportamento através da representação visual.*

1. Introdução

O Problema da Árvore de Steiner é um dos problemas clássicos da teoria dos grafos e da otimização combinatória, com aplicações em diversas áreas como redes de computadores, projeto de circuitos integrados, biologia computacional e logística [Hakimi 1971]. O problema consiste em encontrar uma subárvore de custo mínimo que conecte um conjunto de vértices terminais em um grafo ponderado, podendo incluir vértices adicionais (chamados de pontos de Steiner) para reduzir o custo total da conexão.

Formalmente, dado um grafo $G = (V, E)$ com uma função de custo $c : E \rightarrow \mathbb{R}^+$ e um conjunto de terminais $T \subseteq V$, o objetivo é encontrar uma subárvore $S = (V_S, E_S)$ de G tal que $T \subseteq V_S$ e o custo total $\sum_{e \in E_S} c(e)$ seja minimizado. Este problema é NP-completo [Karp 1972], o que significa que não existem algoritmos conhecidos que o resolvam em tempo polinomial, a menos que $P = NP$.

Devido à complexidade computacional do problema, diversas abordagens heurísticas foram desenvolvidas para encontrar soluções aproximadas em tempo razoável. Entre as heurísticas mais conhecidas estão o algoritmo de Takahashi-Matsuyama

[Takahashi and Matsuyama 1980] e o Shortest Path Heuristic (SPH), que serão abordados neste trabalho.

A visualização de grafos desempenha um papel fundamental na compreensão de algoritmos de grafos complexos. Ferramentas interativas permitem que estudantes e pesquisadores observem o comportamento dos algoritmos passo a passo, facilitando a identificação de padrões e a compreensão de conceitos abstratos [Ellson et al. 2024]. A visualização em três dimensões adiciona uma nova perspectiva, permitindo a exploração de grafos com estruturas mais complexas e a representação de informações adicionais através da profundidade espacial.

Neste contexto, o presente trabalho tem como objetivo desenvolver uma ferramenta interativa de visualização de grafos em 3D para exploração de algoritmos heurísticos de Árvore de Steiner. A ferramenta utiliza a biblioteca Three.js [Cabello et al. 2024] para criar um ambiente imersivo onde usuários podem construir grafos, selecionar terminais e visualizar a execução dos algoritmos Takahashi-Matsuyama e SPH.

Os objetivos específicos deste trabalho são:

- Implementar uma interface interativa para criação e manipulação de grafos em ambiente 3D;
- Desenvolver algoritmos heurísticos para o Problema da Árvore de Steiner;
- Criar mecanismos de visualização que permitam acompanhar a execução dos algoritmos;
- Permitir a comparação entre diferentes abordagens heurísticas;
- Fornecer feedback em tempo real sobre o desempenho e qualidade das soluções encontradas.

A relevância deste trabalho reside na combinação de conceitos teóricos de teoria dos grafos com técnicas modernas de visualização interativa. A ferramenta desenvolvida pode ser utilizada tanto para fins educacionais, auxiliando no ensino de algoritmos de grafos, quanto para fins de pesquisa, permitindo a exploração de novas heurísticas e a análise de seu comportamento em diferentes cenários.

Este artigo está organizado da seguinte forma: a Seção 2 apresenta o desenvolvimento do trabalho, incluindo a metodologia, a implementação dos algoritmos e os aspectos da visualização; a Seção 3 apresenta as conclusões e considerações finais; e as referências bibliográficas são apresentadas ao final.

2. Desenvolvimento

2.1. Metodologia

A metodologia adotada para o desenvolvimento da ferramenta de visualização de grafos em 3D seguiu uma abordagem iterativa, dividida em quatro fases principais: especificação, implementação, testes e refinamento.

Na fase de especificação, foram definidos os requisitos funcionais e não funcionais da ferramenta. Os requisitos funcionais incluíram a capacidade de criar e manipular grafos, selecionar terminais, executar algoritmos heurísticos e visualizar os resultados.

Os requisitos não funcionais focaram na usabilidade, performance e extensibilidade do sistema.

A fase de implementação foi conduzida utilizando JavaScript como linguagem principal, com a biblioteca Three.js para renderização 3D e manipulação de cenas gráficas. A escolha do JavaScript se deve à sua ampla adoção em aplicações web e à facilidade de integração com bibliotecas de visualização. A biblioteca Three.js foi selecionada por sua robustez, documentação extensa e comunidade ativa.

A fase de testes envolveu a validação dos algoritmos implementados através de comparação com resultados conhecidos na literatura, bem como testes de usabilidade com usuários para identificar pontos de melhoria na interface. A fase de refinamento incorporou os feedbacks obtidos, resultando em melhorias na experiência do usuário e na eficiência dos algoritmos.

2.2. Implementação da Visualização 3D

A visualização 3D foi implementada utilizando a biblioteca Three.js, que fornece uma abstração de alto nível sobre a API WebGL do navegador. A cena é composta por três elementos principais: vértices, arestas e o ambiente de navegação.

2.2.1. Vértices

Os vértices são representados como esferas vermelhas posicionadas em coordenadas inteiras do espaço 3D. Cada vértice possui um identificador único e pode ser selecionado pelo usuário através de cliques. A implementação utiliza a classe `SphereGeometry` do Three.js para criar a geometria da esfera e `MeshStandardMaterial` para definir sua aparência.

Para facilitar a identificação dos vértices, cada um possui um rótulo numérico que indica seu identificador. Os rótulos são implementados como sprites texturizados que acompanham a posição do vértice, garantindo que permaneçam visíveis mesmo quando o usuário navega pela cena.

2.2.2. Arestas

As arestas são representadas como cilindros brancos que conectam pares de vértices. A implementação utiliza a classe `CylinderGeometry` do Three.js, com transformações geométricas para posicionar e orientar o cilindro corretamente entre os dois vértices conectados.

Em grafos direcionados, as arestas possuem uma ponta de flecha que indica a direção da conexão. A implementação cria um cone adicional posicionado na extremidade da aresta, orientado na direção do vértice de destino.

Para evitar sobreposição visual quando duas arestas conectam os mesmos vértices em direções opostas (em grafos não direcionados), a implementação aplica um deslocamento lateral às arestas, criando um efeito de curvatura que melhora a legibilidade.

2.2.3. Ambiente de Navegação

O ambiente de navegação permite que o usuário explore o grafo em 3D através de controles de órbita. A implementação utiliza a classe `OrbitControls` do Three.js, que fornece funcionalidades de rotação, zoom e pan da câmera.

Para auxiliar na orientação espacial, a cena inclui um grid auxiliar e eixos de coordenadas. O grid é posicionado no plano XZ e pode ser movido verticalmente para acompanhar a altura dos vértices. Os eixos de coordenadas indicam as direções positivas de X , Y e Z com cores vermelha, verde e azul, respectivamente.

2.3. Algoritmos Heurísticos para Árvore de Steiner

2.3.1. Algoritmo de Takahashi-Matsuyama

O algoritmo de Takahashi-Matsuyama [Takahashi and Matsuyama 1980] é uma heurística clássica para o Problema da Árvore de Steiner. O algoritmo constrói a solução iterativamente, conectando os terminais um por um através de caminhos de menor custo.

A ideia principal do algoritmo é iniciar com um terminal arbitrário e, em cada iteração, encontrar o terminal não conectado mais próximo da árvore atual. A conexão é feita através do caminho de menor custo entre o terminal e a árvore, que é encontrado utilizando o algoritmo de Dijkstra [Dijkstra 1959].

O pseudocódigo do algoritmo é apresentado a seguir:

Algoritmo 1: Algoritmo de Takahashi-Matsuyama

Entrada: Grafo $G = (V, E)$, conjunto de terminais T

Saída: Árvore de Steiner S

```
1  $T' \leftarrow T$ ;  
2  $S \leftarrow \{t_1\}$ , onde  $t_1 \in T$ ;  
3  $T' \leftarrow T \setminus \{t_1\}$ ;  
4 enquanto  $T' \neq \emptyset$  faça  
5    $melhorCaminho \leftarrow \infty$ ;  
6    $terminalAlvo \leftarrow \text{nulo}$ ;  
7   para cada  $t \in T'$  faça  
8      $caminho \leftarrow \text{Dijkstra}(t, S)$ ;  
9     se  $\text{custo}(caminho) < melhorCaminho$  então  
10        $melhorCaminho \leftarrow caminho$ ;  
11        $terminalAlvo \leftarrow t$ ;  
12    $S \leftarrow S \cup \text{vertices}(melhorCaminho)$ ;  
13    $T' \leftarrow T' \setminus \{terminalAlvo\}$ ;  
14 retorna  $S$ ;
```

A complexidade de tempo do algoritmo é $O(k \cdot (E + V \log V))$, onde k é o número de terminais, V é o número de vértices e E é o número de arestas. Esta complexidade deriva da execução do algoritmo de Dijkstra para cada terminal não conectado.

2.3.2. Shortest Path Heuristic (SPH)

O Shortest Path Heuristic (SPH) é uma variação do algoritmo de Takahashi-Matsuyama que, em vez de conectar apenas terminais, considera todos os vértices do grafo como candidatos para expansão da árvore. Esta abordagem pode resultar em soluções de melhor qualidade, pois permite a inclusão de pontos de Steiner intermediários que reduzem o custo total.

A ideia principal do SPH é encontrar, em cada iteração, o vértice mais próximo da árvore atual, independentemente de ser um terminal ou não. O caminho de menor custo entre este vértice e a árvore é adicionado à solução, e o processo é repetido até que todos os terminais estejam conectados.

O pseudocódigo do algoritmo é apresentado a seguir:

Algoritmo 2: Shortest Path Heuristic (SPH)

Entrada: Grafo $G = (V, E)$, conjunto de terminais T
Saída: Árvore de Steiner S

```
1  $T' \leftarrow T$ ;  
2  $S \leftarrow \{t_1\}$ , onde  $t_1 \in T$ ;  
3  $T' \leftarrow T \setminus \{t_1\}$ ;  
4 enquanto  $T' \neq \emptyset$  faça  
5    $melhorCaminho \leftarrow \infty$ ;  
6    $verticeAlvo \leftarrow \text{nulo}$ ;  
7   para cada  $v \in V \setminus S$  faça  
8      $caminho \leftarrow \text{Dijkstra}(v, S)$ ;  
9     se  $\text{custo}(caminho) < melhorCaminho$  então  
10       $melhorCaminho \leftarrow caminho$ ;  
11       $verticeAlvo \leftarrow v$ ;  
12    $S \leftarrow S \cup \text{vertices}(melhorCaminho)$ ;  
13   se  $verticeAlvo \in T'$  então  
14      $T' \leftarrow T' \setminus \{verticeAlvo\}$ ;  
15 retorna  $S$ ;
```

A complexidade de tempo do SPH é $O(V \cdot (E + V \log V))$, onde V é o número de vértices e E é o número de arestas. Esta complexidade é maior que a do algoritmo de Takahashi-Matsuyama, pois o SPH considera todos os vértices em cada iteração, não apenas os terminais.

2.3.3. Algoritmo de Dijkstra

Ambas as heurísticas utilizam o algoritmo de Dijkstra [Dijkstra 1959] para encontrar caminhos de menor custo. O algoritmo de Dijkstra é um algoritmo clássico de caminhos mínimos em grafos ponderados com pesos não negativos.

A implementação utiliza uma fila de prioridade para selecionar o vértice com menor distância não processado. Para cada vértice processado, o algoritmo relaxa as arestas

incidentes, atualizando as distâncias dos vértices vizinhos quando um caminho mais curto é encontrado.

A complexidade de tempo do algoritmo de Dijkstra é $O(E + V \log V)$ quando implementado com uma fila de prioridade baseada em heap binário, onde V é o número de vértices e E é o número de arestas.

2.4. Funcionalidades da Ferramenta

A ferramenta desenvolvida oferece diversas funcionalidades para criação, manipulação e análise de grafos. A seguir, são descritas as principais funcionalidades implementadas.

2.4.1. Criação e Manipulação de Grafos

A criação de grafos é realizada através de uma interface intuitiva baseada em cliques. O usuário pode criar vértices clicando em posições vazias do espaço 3D. A implementação utiliza raycasting para determinar a posição do clique no espaço 3D e aplica um "snap" para coordenadas inteiras, facilitando o alinhamento dos vértices.

A criação de arestas é realizada através de cliques em pares de vértices. Quando o usuário clica em um vértice e depois em outro, uma aresta é criada conectando os dois vértices. Em grafos direcionados, a direção da aresta segue a ordem dos cliques.

A manipulação de grafos inclui funcionalidades de mover vértices, deletar elementos e alternar entre modos de grafo. O usuário pode mover vértices utilizando as teclas de seta quando o modo de movimento está ativado. A deleção de elementos é realizada através da tecla Delete, removendo o vértice ou aresta selecionado.

2.4.2. Modos de Grafo

A ferramenta suporta grafos direcionados e não direcionados, bem como grafos ponderados e não ponderados. O modo direcionado pode ser alternado através da tecla 'd', enquanto o modo ponderado pode ser alternado através da tecla 'p'.

Em grafos ponderados, as arestas possuem pesos que são considerados nos cálculos de custo. Os pesos podem ser editados através da tecla 'e', que abre um prompt para inserir o novo valor. A visualização dos pesos é ativada automaticamente quando o modo ponderado está ativo.

2.4.3. Algoritmos de Busca

A ferramenta implementa algoritmos de busca em largura (BFS) e busca em profundidade (DFS) para exploração do grafo. Os algoritmos são executados a partir do vértice selecionado e apresentam uma animação visual que mostra a ordem de visita dos vértices.

A animação utiliza cores diferentes para indicar o estado de cada vértice: vermelho para vértices não visitados, laranja para o vértice sendo processado e verde para vértices já visitados. As arestas também são coloridas durante a animação, com azul para BFS e rosa para DFS.

2.4.4. Seleção de Terminais

A seleção de terminais para os algoritmos de Árvore de Steiner é realizada através do modo de terminais, ativado pela tecla 't'. Neste modo, o usuário pode selecionar múltiplos vértices como terminais clicando neles. Os terminais são destacados visualmente com uma cor diferente para facilitar sua identificação.

2.4.5. Execução e Comparação de Heurísticas

A execução dos algoritmos heurísticos é realizada através das teclas '1' e '2', que executam o algoritmo de Takahashi-Matsuyama e o SPH, respectivamente. Após a execução, a ferramenta apresenta um resumo com o tempo de execução, custo total, número de vértices e número de arestas da solução encontrada.

A comparação entre os algoritmos pode ser realizada através da tecla '3', que apresenta uma tabela comparativa com os resultados das duas heurísticas. A ferramenta armazena os resultados das execuções anteriores, permitindo comparações mesmo após modificações no grafo.

2.4.6. Diagnóstico e Restauração

A ferramenta inclui funcionalidades de diagnóstico e restauração do estado do grafo. O diagnóstico, ativado pela tecla 'i', apresenta informações sobre a conectividade do grafo, número de vértices isolados e outras métricas relevantes.

A restauração do estado original, ativada pela tecla 'r', permite que o usuário reverta as modificações realizadas pelos algoritmos heurísticos, facilitando a exploração de diferentes abordagens sem a necessidade de reconstruir o grafo.

2.4.7. Importação e Exportação

A ferramenta suporta importação e exportação de grafos através de arquivos JSON. A exportação é realizada através da combinação Ctrl+S, enquanto a importação é realizada através da combinação Ctrl+I. Esta funcionalidade permite que usuários salvem e compartilhem grafos criados, facilitando a reprodução de experimentos e a colaboração entre usuários.

2.5. Resultados e Discussão

A ferramenta desenvolvida demonstrou ser eficaz na visualização e exploração de algoritmos heurísticos para Árvore de Steiner. A interface intuitiva e a visualização em 3D facilitaram a compreensão do comportamento dos algoritmos, permitindo que usuários identificassem padrões e comparassem diferentes abordagens.

Os algoritmos implementados apresentaram resultados consistentes com a literatura. O algoritmo de Takahashi-Matsuyama demonstrou ser mais rápido que o SPH, enquanto o SPH tendeu a produzir soluções de menor custo em grafos mais densos. Esta

observação está de acordo com as expectativas teóricas, pois o SPH considera um espaço de busca maior.

A visualização em 3D provou ser particularmente útil para grafos com estruturas complexas. A capacidade de navegar pelo grafo em três dimensões permitiu que usuários identificassem padrões que seriam difíceis de perceber em visualizações 2D tradicionais.

A implementação dos algoritmos de busca (BFS e DFS) com animação visual facilitou a compreensão de conceitos fundamentais de teoria dos grafos. A visualização passo a passo permitiu que usuários acompanhassem a ordem de visita dos vértices e compreendessem as diferenças entre as duas abordagens de busca.

As funcionalidades de diagnóstico e restauração demonstraram ser valiosas para a exploração de diferentes cenários. A capacidade de restaurar o estado original do grafo permitiu que usuários experimentassem diferentes configurações de terminais e comparassem os resultados sem a necessidade de reconstruir o grafo.

A ferramenta também demonstrou potencial para aplicações educacionais. A interface intuitiva e o feedback visual em tempo real tornam o aprendizado de algoritmos de grafos mais acessível e engajador. A possibilidade de criar grafos personalizados e experimentar com diferentes algoritmos permite que estudantes desenvolvam uma compreensão mais profunda dos conceitos.

No entanto, algumas limitações foram identificadas. A performance da ferramenta pode degradar em grafos muito grandes, devido à complexidade dos algoritmos implementados e à renderização 3D. Além disso, a interface pode ser melhorada para usuários menos experientes, com tutoriais e guias interativos.

Futuras melhorias podem incluir a implementação de outras heurísticas para o Problema da Árvore de Steiner, como o algoritmo de Robins-Zelikovsky [Robins and Zelikovsky 2000] e o algoritmo de Goemans-Williamson [Goemans and Williamson 1994]. Também seria interessante adicionar funcionalidades de análise de complexidade e comparação com soluções ótimas quando disponíveis.

3. Conclusão

Este trabalho apresentou o desenvolvimento de uma ferramenta interativa de visualização de grafos em 3D para exploração de algoritmos heurísticos de Árvore de Steiner. A ferramenta combina conceitos teóricos de teoria dos grafos com técnicas modernas de visualização interativa, proporcionando um ambiente imersivo para criação, manipulação e análise de grafos.

A implementação dos algoritmos de Takahashi-Matsuyama e Shortest Path Heuristic demonstrou a eficácia das abordagens heurísticas para o Problema da Árvore de Steiner. A visualização em tempo real da execução dos algoritmos permitiu que usuários compreendessem o comportamento de cada heurística e identificassem suas características distintivas.

Os resultados obtidos indicam que a ferramenta é adequada tanto para fins educacionais quanto para fins de pesquisa. A interface intuitiva e o feedback visual facilitam o aprendizado de algoritmos de grafos, enquanto as funcionalidades de comparação e diagnóstico permitem a exploração de diferentes cenários e a análise de resultados.

A visualização em 3D provou ser particularmente valiosa para grafos com estruturas complexas. A capacidade de navegar pelo grafo em três dimensões permitiu a identificação de padrões que seriam difíceis de perceber em visualizações 2D tradicionais, demonstrando o potencial de técnicas de visualização avançadas para a análise de estruturas de grafos.

As funcionalidades implementadas, como criação e manipulação de grafos, seleção de terminais, execução de algoritmos heurísticos, comparação de resultados e diagnóstico de conectividade, proporcionam um conjunto completo de ferramentas para exploração do Problema da Árvore de Steiner. A possibilidade de importar e exportar grafos facilita a reprodução de experimentos e a colaboração entre usuários.

No entanto, algumas limitações foram identificadas. A performance da ferramenta pode degradar em grafos muito grandes, devido à complexidade dos algoritmos implementados e à renderização 3D. Além disso, a interface pode ser melhorada para usuários menos experientes, com tutoriais e guias interativos.

Futuras melhorias podem incluir a implementação de outras heurísticas para o Problema da Árvore de Steiner, como o algoritmo de Robins-Zelikovsky [Robins and Zelikovsky 2000] e o algoritmo de Goemans-Williamson [Goemans and Williamson 1994]. Também seria interessante adicionar funcionalidades de análise de complexidade e comparação com soluções ótimas quando disponíveis.

Outras direções futuras incluem a expansão da ferramenta para suportar outros problemas de otimização em grafos, como o Problema do Caixeiro Viajante e o Problema de Fluxo Máximo. A implementação de algoritmos exatos para o Problema da Árvore de Steiner também seria valiosa, permitindo a comparação entre soluções heurísticas e ótimas.

Em termos de visualização, futuras melhorias podem incluir a implementação de técnicas de realidade virtual e aumentada, proporcionando uma experiência ainda mais imersiva. A adição de funcionalidades de colaboração em tempo real também seria interessante, permitindo que múltiplos usuários trabalhem simultaneamente no mesmo grafo.

Em conclusão, a ferramenta desenvolvida representa uma contribuição significativa para a área de visualização de algoritmos de grafos. A combinação de conceitos teóricos com técnicas modernas de visualização proporciona uma plataforma poderosa para exploração e compreensão de algoritmos heurísticos para Árvore de Steiner. Espera-se que esta ferramenta seja útil tanto para fins educacionais quanto para fins de pesquisa, contribuindo para o avanço do conhecimento na área de teoria dos grafos e otimização combinatória.

Referências

- Cabello, R. et al. (2024). Three.js - javascript 3d library. *GitHub Repository*.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische mathematik*, 1(1):269–271.
- Ellson, J. et al. (2024). Graphviz - graph visualization software. *AT&T Labs Research*.

- Goemans, M. X. and Williamson, D. P. (1994). A 1.598 approximation algorithm for the steiner problem in graphs. *Proceedings of the 6th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1–8.
- Hakimi, S. L. (1971). Steiner’s problem in graphs and its implications. *Networks*, 1(2):113–133.
- Karp, R. M. (1972). Reducibility among combinatorial problems. *Complexity of computer computations*, pages 85–103.
- Robins, G. and Zelikovsky, A. (2000). Improved approximation algorithms for the steiner tree problem. *Journal of Algorithms*, 46(2):125–142.
- Takahashi, H. and Matsuyama, A. (1980). An approximate solution for the steiner problem in graphs. *Mathematica Japonica*, 24(6):573–577.